Preserving semantic continuity across actors: a tell-based approach without orchestration

Alvaro Rivera

2026-07-04

Abstract

This paper is an analytic theory contribution in the sense of Gregor's (2006) *theory for analyzing* (Type I): it identifies a structural defect in the canonical actor-systems literature, derives the design principles required to address it, and presents an instantiation as an existence proof that the alternative is realizable — not the design-science evaluation of an artifact (Hevner, March, Park, & Ram, 2004), which a design-science case study would undertake separately. For five decades, from Carl Hewitt's original formulation through Gul Agha, Joe Armstrong, and modern frameworks such as Akka and Microsoft Orleans, cross-actor causation has been treated as an operational concern of the runtime rather than as a construct of the program. As a result, the program of an actor is structurally fragmented at every actor boundary: when one actor causes an effect in another, the causal sentence disappears from the program and reappears as infrastructure. This paper shows that this fragmentation is not inherent to the actor model but a contingent design choice. It derives the design principles under which cross-actor message passing can be expressed as a sentence of the actor's program without violating actor isolation and without introducing orchestration. An existing primitive, tell, is shown to satisfy these principles when recorded — as a dense program operation rather than a serialized payload — in the actor's journal. Because that record is an assertion of a fact the sender has lived, it is named in the past tense, distinct from a command in the present imperative — the verb tense marks which act the sentence performs, a distinction the operational treatment's single dispatch verb erases. The realization presupposes that journal: the contribution is the cross-actor extension of a journal-as-program substrate, not the primitive in isolation. Under this construction, program continuity extends across actors, edge by edge — each send recorded as program in its sender's journal — while preserving the defining properties of actor systems.

Preserving semantic continuity across actors

TL;DR

The actor model treats cross-actor causation as operational — message dispatch by a runtime, not a statement in any actor's program. The treatment is not entailed by the model; it is a contingent design decision, adopted as the default frame of the canonical actor lineage and seldom examined as a choice. The cost is structural: an entire ecosystem of compensating patterns — sagas, choreography, distributed tracing, workflow engines — exists to reconstruct cross-actor causal chains that no actor's program records. The treatment even erases a grammatical distinction — one dispatch verb collapsing an assertion of fact (past tense) and a command (present imperative) into a single send — which the programmatic alternative restores.

This paper observes that the assumption can be removed without altering any structural commitment of the actor model. Three conditions describe a system in which the cross-actor send is recorded as a sentence in the sender's program: locality of writes (each journal records only its own actor's activity), causation as program statement (the cross-actor send is a sentence in the sender's journal), and no external coordinator (no party outside the participating actors decides what happens next). Any primitive satisfying these three conditions dissolves the assumption — on the dense journal-as-program substrate the conditions presuppose (the send recorded as a sentence, not a payload). *Tell* — a primitive in the Puppeteer framework — is one such realization.

Three implementations of the same domain — saga, choreography, tell — are exhibited side by side. The reader sees, in journals, that the saga places the joint history in a coordinator's program, that the choreography places it in an external bus log, and that the tell instantiation places it in the sender's own program. Four tests probe these claims under tell. Three show that the journal alone supports replay reconstruction, cross-datacenter replication, and audit query — operations that require external apparatus under the assumption. The fourth shows that after a crash in the dispatch window the journal still records a stranded tell's fate honestly, recovered from the transport's testimony rather than fabricated. Each cross-actor edge is recorded as program in its sender's journal; a multi-hop chain composes these per-edge records — programmatic at every hop, where the alternatives reconstruct the chain from artifacts outside any program (§8.5). **Fifty years of convention are not fifty years of necessity.**

Claims this paper makes

1. **The assumption named.** Cross-actor causation has been treated as operational rather than programmatic across fifty years of canonical actor-systems literature — from Hewitt et al. (1973) through Akka and Orleans. *(Verification: §2 + the genealogy table in §2.4.)*

2. **The assumption is contingent.** No theorem of the actor model entails it. The three structural commitments of the model — autonomy, message-based communication, isolation — define the ontology of actors, not the mechanics of their runtimes. *(Verification: §4.)*

3. **Three conditions resolve the assumption.** Locality of writes (C1), causation as program statement (C2), and no external coordinator (C3) describe a system in which semantic continuity is preserved across actors without violating any of the actor model's commitments. The conditions are not design choices; they are the consequences of reconciling the model's commitments with the construct. *(Verification: §7.1.)*

4. **Any primitive satisfying C1+C2+C3 dissolves the assumption — given a journal-as-program substrate.** The realization is one of many possible, but the conditions are not free-floating: C2 requires the send to be recorded as a *statement of the program* — a dense operation, not a serialized event — which presupposes the anti-porosity of Paper 1 and the externalized parameters of Paper 2. The claim is therefore *tell on such a substrate*, not *tell in any actor framework*; a framework that records sends as opaque payloads would satisfy C2 only by first adopting that substrate, which is the subject of Papers 1–2 and beyond this paper's scope. *(Verification: §7.5.)*

5. **The actor model's commitments survive the reformulation intact.** Autonomy, message-based communication, and isolation each remain unchanged; only the historical interpretation of what each actor's program is *permitted to say* is removed. *(Verification: §7.3.)*

6. **The compensating ecosystem exists because of the assumption.** Saga orchestrators, event-driven choreography, distributed tracing, and workflow engines each compensate, in their own way, for the absence of cross-actor causation in any actor's program. Their sophistication, maturity, and widespread adoption are evidence of the cost of the assumption, not of its correctness. The scope is the *record* of cross-actor causation: tell relocates that record into the program; it does not provide the transactional recovery — compensation, rollback, timeouts — that sagas and workflow engines also carry, and does not claim to (§6.1). *(Verification: §6 + the side-by-side labs in §8.3.)*

7. **Tell exhibits the conditions in a runnable instantiation.** A Reaction whose `.Causation.Continue(...)` body issues a `tell` records the cross-actor assertion in the sender's journal — as a typed message-action

— and the receiver's acknowledgment closes the round-trip. The sender's journal becomes a self-contained record of that cross-actor edge; a multi-hop chain composes these per-edge records — programmatic at every hop, unlike the alternatives' reconstruction from outside any program (§8.5). *(Verification: §8.2 + §8.3 Style 3 + §8.5.)*

8. **Auditing the cross-actor narrative becomes reading the program.** The journal shows what was sent, to whom, with what content, at what time, and with what acknowledgment — without correlation IDs, distributed tracing, or external aggregation. *(Verification: §8.5 G3.)*

9. **Replay reconstructs cross-actor state from the journal alone — and recovers each tell's fate.** A fresh actor with no shared transport and no live receiver, replaying the journal, reconstructs the in-flight tell state; for a tell stranded by a crash in the dispatch window, the rehydrated actor reconstructs the pending tell and the transport testifies its fate, which the journal records as a verdict — acked, not-delivered, or honestly pending — so recovery never leaves the journal asserting a send that did not land. *(Verification: §8.5 G1, G4.)*

10. **Cross-datacenter replication preserves the cross-actor causal chain.** The journal carries the cross-actor causation across data centers because the causation was always recorded in a place replication can carry. *(Verification: §8.5 G2.)*

---

1. Introduction

When one actor causes another to act, where does that causal sentence live? Consider the ordinary case. An actor performs an operation and, as a result, sends a message to another actor. The first actor's operation, state mutation, and emitted events appear in its journal or trace; the second receives the message, and its own operation is likewise recorded. Yet the act of sending — the causal step that connects the two — appears in neither actor's program. It is mediated by the runtime, the dispatcher, or the message broker: the sender's journal does not record that it spoke, and the receiver's journal does not record, in program terms, who spoke to it. Ask later why the second actor did what it did, and the answer has to be assembled from outside every program — partly from the first actor's log, partly from the second's, partly from the broker's offsets, partly from a distributed trace. The causal chain exists. No program contains it.

This is so familiar that it rarely appears noteworthy. But it should. If actors are programs, and a sentence in one actor's program causes an effect in another, then that causal sentence has every reason to appear as part of the first actor's program. That it does not is no logical consequence of the actor model; it is a structural feature of how actor systems have historically been constructed.

This paper names that gap. The construct introduced is *semantic continuity*: the property of a program (in the substrate-level sense established in Paper 2 §1.2: the pair of domain library and journal of invocations) whose causal structure remains recorded as part of the program itself, even when its effects cross boundaries. The defect identified is the absence of semantic continuity at actor boundaries. The principles derived are the conditions under which it can be preserved without violating actor isolation and without introducing orchestration. The instantiation presented is *tell*, a primitive in which the cross-actor send is recorded as a sentence of the sender's journal — a dense program operation, not a serialized payload, which presupposes the anti-porous journal established in Paper 1.

Methodologically, this is an analytic theory contribution: it names a structural assumption that the canonical literature on actor-based systems has documented in different forms without recognizing as a single construct, derives the principles under which the assumption can be rejected, and presents a running system in which those principles are realized as an existence proof — not as the substance of the claim. The weight of the argument is conceptual: the naming of the assumption (§3) and the account of why an entire ecosystem of patterns answers it (§6) carry the claim, while the instantiation (§8) is its existence proof. The genre is the one Gregor (2006) names *theory for analyzing* (Type I): it introduces a construct that lets the phenomenon be described and classified, with empirical evaluation supplementary; the Hevner-style design-science *evaluation* of the artifact (Hevner, March, Park, & Ram, 2004) is a separate undertaking this paper does not attempt. The conditions C1–C3 derived in §7.1 are the necessary conditions the construct entails, not a prescriptive method offered for adoption: deriving what a system must satisfy to preserve the construct is Type I analysis, where prescribing and evaluating an artifact would be a separate design-science (Type V) contribution this paper does not undertake. This is not a systems paper — it presents no performance benchmarks, fault-injection metrics, or latency comparisons against existing actor frameworks; analytic theory measures contribution by the precision of the construct, the validity of the principles, and the realizability of the instantiation. Readers expecting quantitative comparisons against alternatives will find structural comparisons — what each pattern records, where the joint history lives — in §6 and §8.4.

§2 traces the genealogy of the assumption that produces this gap, showing that across five decades and multiple canonical generations of literature the assumption has remained continuous in substance though varied in form. §3 names the assumption explicitly: *causation between actors is treated as operational rather than programmatic*. §4 demonstrates that this assumption is contingent rather than necessary — no theorem of the actor model entails it. §5 examines the architectural and operational consequences of the assumption. §6 shows why existing responses to those consequences — sagas, choreography, distributed tracing, and workflow engines — cannot dissolve the assumption, because they reconstruct cross-actor flow after the fact rather than preserving it as program. §7 reformulates the model: under three explicit conditions, semantic continuity

can be preserved across actors. §8 presents the instantiation, *tell*, through an illustrative case study against saga and choreography implementations of the same domain. §9 relates the construct to prior work in this paper series. §10 concludes.

---

2. Genealogy

2.1 The founding decision (1970s)

The actor model was introduced in 1973 by Hewitt, Bishop, and Steiger as a unifying account of concurrent computation in which "all of the modes of behavior can be defined in terms of one kind of behavior: sending messages to actors" (Hewitt et al., 1973, p. 235). Sending messages is positioned as a universal primitive, but the framing goes further: it is positioned as a *machine-level* primitive. "The basic unit of execution on an actor machine is sending a message in much the same way that the basic unit of execution on present day machines is an instruction" (Hewitt et al., 1973, pp. 236–237). The act of sending is therefore parallel to a hardware instruction — it is what the abstract machine performs *between* actors, not content of any actor's program.

A different concurrency tradition emerged in 1978 with Hoare's *Communicating Sequential Processes* (Hoare, 1978). It is worth distinguishing rather than assimilating: CSP's communication is a *symmetric, synchronous* handshake at named ports — sender and receiver rendezvous, each blocking until the other is ready — which is the opposite of the actor model's *asynchronous, fire-and-forget* send. These are different ontologies, and this paper does not fold them together. CSP earns one narrow point of contrast: there too, cross-process communication is a construct of the language, not a statement in either process's program. But the genealogy below tracks the actor lineage specifically; CSP is a foil, not a second witness to the same assumption.

The actor model was formalized in 1986 in Agha's MIT thesis (Agha, 1986), which defined actors as behavior functions over messages and treated cross-actor effects as emitted output messages of the function. The formal account ratified what Hewitt et al. (1973) had stated informally: an actor's program describes how it responds to the messages it receives; what happens *between* actors is the operational semantics of the model, not the content of any actor's program.

By the close of the 1980s, the actor lineage — Hewitt's informal formulation, Agha's formalization — had stabilized a single architectural decision: the boundary between an actor and the message-passing layer was drawn deliberately, and the layer between actors was characterized as operational rather than programmatic. (CSP reached a structurally similar boundary by a different route and under a different ontology; the resemblance is worth a note of contrast, not a claim that one assumption spans both traditions.)

2.2 The pragmatic maturation (1990s–2000s)

The operational frame became productive in Erlang. Armstrong's (2003) PhD thesis argued that fault tolerance becomes tractable precisely because processes do not share programmatic flow: failure is handled across process boundaries by links and exit signals. In Armstrong's own terms, "a process can supervise the existence of another process by setting up a link to it. When a process terminates, it automatically sends exit signals to the process to which it is linked" (Armstrong, 2003, p. 217). Supervision is a structural pattern in which a supervisor observes the failure of a child process and reacts. The supervisor's program is local; the failed process's program does not extend into the supervisor; the cross-process relationship is mediated by VM-level links and exit signals. Cross-process causation is an infrastructure concern. The frame stabilized further by becoming useful: the operational-rather-than-programmatic separation was now what made fault-tolerant systems possible.

2.3 The modern instantiations (2010s–)

Two implementations carried the actor frame into modern production systems. Akka, the canonical JVM actor framework (Lightbend, n.d.), characterizes its fundamental message-send pattern directly:

> *"Tell is asynchronous which means that the method returns right away. After the statement is executed there is no guarantee that the message has been processed by the recipient yet. It also means there is no way to know if the message was received, the processing succeeded or failed."*

The verb is named — `tell` — and so is its absence of program-level effect. The dispatch occurs; the sender's program does not record that it occurred, nor whether the recipient acted on it.

Microsoft's Orleans (Bernstein et al., 2014) introduced *virtual actors*: location-transparent grains whose dispatch is mediated by the Orleans runtime. Each grain has a key; the runtime decides where the grain lives and routes calls; the grain's code observes only local state and method invocation. The runtime maintains the cross-grain dispatch logic as infrastructure; the grain's program never sees nor records cross-grain causation as part of its narrative.

> **Decades apart, one assumption.** In Hewitt et al. (1973): *"Sending a message to an actor makes no presupposition that the actor sent the message will ever send back a message to the continuation"* (p. 241). In Akka 2014–present: *"there is no way to know if the message was received, the processing succeeded or failed."* The two formulations are decades apart and identical in substance. The verb `tell` has been part of actor-systems vocabulary throughout. What this paper observes is that the verb names the dispatch, not the program: a `tell` from actor A to actor B leaves no trace in either ac-

tor's program. The verb has existed; the assumption that the verb's effect was extra-programmatic has survived intact alongside it.

2.4 The naturalized assumption

Across fifty years and four canonical generations of literature — Hewitt's foundational paper, Agha's formalization, Armstrong's pragmatic maturation, and the modern instantiations in Akka and Orleans — the question of whether cross-actor flow is a *statement* of any actor's program is not the one the lineage asks. Where cross-entity causation is recorded at all — the message logs and causation identifiers of §6.4 — it is recorded operationally, beneath or beside the program, never as the program's own sentence. The assumption does not appear as a defended thesis; it is the default frame within which the lineage operates.

The forms in which the assumption surfaces vary by generation; the substance is continuous.

Year	Work	Anchor	Form of the assumption
1973	Hewitt et al. — *A Universal Modular ACTOR Formalism* (IJCAI)	"The basic unit of execution on an actor machine is sending a message in much the same way that the basic unit of execution on present day machines is an instruction." (pp. 236–237)	Sending is a machine-level primitive, parallel to a hardware instruction; therefore not content of any actor's program.
1986	Agha — *Actors: A Model of Concurrent Computation* (MIT)	Actors are behavior functions; cross-actor effects are emitted output messages of the function.	The behavior function is local; the cross-actor effect is the *output* of the function, separate from the function's body.

Year	Work	Anchor	Form of the assumption
2003	Armstrong — *Making reliable distributed systems…* (KTH)	"A process can supervise the existence of another process by setting up a link to it […] it automatically sends exit signals to the process to which it is linked." (p. 217)	Cross-process flow is infrastructure-level (VM links and exit signals), not part of any process's program.
2010s	Lightbend — Akka *Interaction Patterns*	"There is no way to know if the message was received, the processing succeeded or failed."	The dispatch verb (`tell`) exists; its effect is explicitly outside the sender's program — no acknowledgment record.
2014	Bernstein et al. — *Orleans: Distributed Virtual Actors*	The Orleans runtime places grains and mediates cross-grain calls; grain code sees only local state.	Cross-grain dispatch is hidden by the runtime; the grain's program never sees nor records cross-grain causation.

The continuity of this assumption across five decades is not the result of oversight, and still less of error. Each canonical contribution was optimizing a different property: Hewitt, Bishop, and Steiger, a uniform account of concurrency; Agha, a formal semantics of actor behavior; Armstrong, fault tolerance through process isolation; Akka and Orleans, distribution and location transparency. None was optimizing the *programmatic preservation of cross-actor causation*, because that was not the property under design. The assumption is the shadow of a different objective function, not a mistake the field failed to notice — which is why questioning it is not a charge against the lineage but a change in what one chooses to optimize. What follows does not dispute the effectiveness of the separation; it questions whether the separation is necessary.

§3 names this assumption explicitly. §4 demonstrates that it is contingent rather

than necessary.

---

3. The assumption named

What §2 has shown to be continuous across fifty years admits a single one-line formulation. The structural claim that links the five entries of §2.4 — Hewitt's "machine-level instruction", Agha's emitted output messages, Armstrong's VM-supported links, Akka's `tell`, Orleans' runtime-mediated dispatch — is:

Causation between actors is operational, not programmatic.

The two terms in the formulation are the working categories of the rest of this paper. They also answer, for now, the question that opened it: *where does the causal sentence live?* Under the assumption, it lives in the operational layer — between actors, mediated by the runtime — and not in any actor's program.

Operational designates effects that the system performs around or between actors but that no actor's program records. Message dispatch by a runtime, supervision links between processes, virtual-actor placement decisions, message-broker routing, and distributed-tracing correlation IDs are all operational in this sense: their existence is mediated by infrastructure, and their effect on the system is real, but no program in the system contains the act of mediation as a statement.

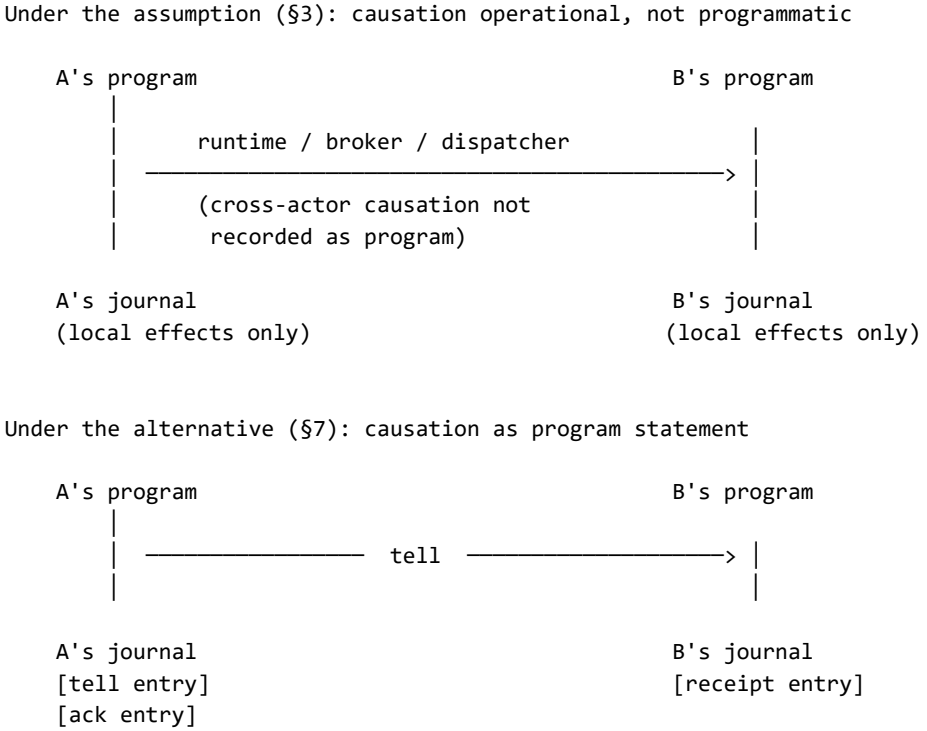
Programmatic designates an effect that an actor's program contains as a *statement of the program* — an operation the program executes, observable within the program's own narrative and re-executed when the program is replayed. A method invocation an actor performs on itself is programmatic in this sense; so is a state mutation it writes. The program records what it did, as the doing, and replay re-runs it.

The distinction has a linguistic signature. A *command* is a directive — it asks for what is not yet done, named in the imperative present (`ApplyReward`, `Confirm`); an *assertion* reports a fact already lived, named in the past (`PurchaseConfirmed`). Treating the cross-actor send as operational collapses the two into one runtime dispatch — a single send verb carrying both, as in the canonical frameworks (§2.4); treating it as programmatic keeps them distinct and lets the sender record the one it can truthfully make — the assertion of what it did. The verb tense is the surface mark of which act the sentence performs; the choice this paper argues for is the one under which that mark survives into the program (developed at §8.2).

The boundary is not whether the send is recorded in the actor's own log. A serialized event can be appended there without being a statement of the program. Conventional event sourcing already does this: an aggregate raises a `MessageSent` or integration event when it dispatches to another context, and the event lands in the aggregate's own stream. But that event is *data the program emitted*,

not a *sentence the program runs* — on replay it is folded back into state by a reducer, never re-executed as a statement of the program, and the dispatch is relayed by an outbox or broker the program does not contain. By the test used here it is operational-in-effect despite its location. A recorded cross-actor send is *programmatic* only when the send is a dense operation in the program — an executable sentence naming its recipient and message by reference, replayed as program rather than re-applied as state. Replayed *as program* means the recorded statement is re-executed by the interpreter — the program re-runs, reconstructing the state the send produced — not that the external dispatch is repeated: no journaled operation repeats its external effects on replay (replaying a debit reconstructs the balance without re-contacting the bank), and the tell is no exception. The programmatic property is that the send is a re-executable statement the program re-runs, where a serialized event is data a reducer folds without the program ever re-executing a send. That density criterion — a recorded operation, not a serialized payload — is the *anti-porosity* established in Paper 1; the present paper takes it as a precondition and asks what becomes possible at the actor boundary once it holds.

The contrast can be visualized:



Both views describe the same flow; they differ in what is recorded as program. Under the assumption, the cross-actor send lives in infrastructure between actors and is invisible from any actor's program. Under the alternative, the cross-actor

send is itself a journal entry on the sender's side, with a corresponding receipt entry on the receiver's side.

The naturalized assumption is the claim that, in the ontology of an actor system, the act of one actor causing an effect in another belongs to the *operational* category and not the *programmatic* one. The 1973 framing of message-sending as a hardware-instruction analogue is this claim. The 2014–present framing of `tell` as a fire-and-forget dispatch with no acknowledgment record is this claim. Across formulations, the assumption holds.

§4 demonstrates that the assumption, while continuous in the literature, is contingent rather than necessary: no theorem of the actor model entails it. §5 examines the consequences of the assumption — what is structurally lost when cross-actor causation lives outside any program. §6 shows why the existing repertoire of patterns built atop the actor model — sagas, choreography, distributed tracing, workflow engines — cannot dissolve the assumption, because they reconstruct cross-actor flow *after* the fact rather than preserving it as program. §7 reformulates: the assumption is contingent, the alternative is constructible, and the cross-actor flow can be a sentence of the program.

---

4. Contingency

The assumption named in §3 — that causation between actors is treated as operational rather than programmatic — would be necessary if it followed from a theorem of the actor model, from a foundational property the model formally entails, or from the structural commitments that distinguish actor systems from other concurrency paradigms. It does not. This section traces what the actor model formally requires and what it does not, and shows that the assumption is a contingent feature of how actor systems have historically been built rather than a consequence of what they are.

4.1 What the actor model entails

The actor model, in its canonical formulations from Hewitt et al. (1973) through Agha (1986), formally entails three structural commitments:

1. **Autonomy.** Each actor has its own state and processes messages serially. No actor's behavior is contingent on the simultaneous execution of another's.
2. **Message-based communication.** Actors communicate by passing messages. No actor reads or writes another's state directly.
3. **Isolation.** An actor's state is private to it. The address space of an actor is not shared.

These three commitments are what distinguish actor systems from shared-memory concurrency models. They define the ontology of actors, not the me-

chanics of their runtimes. They are the substance of "what the actor model is" in any reading of the canonical sources.

4.2 What it does not entail

The three commitments above do not entail that the *act of sending* be invisible to the sender's own program. They do not entail that cross-actor dispatch be mediated by a runtime layer that owns the send. They do not entail that a record of "actor A spoke to actor B at time T" must reside outside actor A's narrative.

In Hewitt et al. (1973), the act of sending is an action the actor performs. The framing of message-sending as a machine-level instruction analogous to a hardware instruction (Hewitt et al., 1973, pp. 236–237) is an interpretive choice that supports the architecture proposed in that paper; it is not a formal consequence of what an actor is. In Agha (1986), the actor's behavior is modelled as a function from (state, message) to (next state, outgoing messages) (Agha, 1986, ch. 4). The outgoing messages are output of the function. The formalization neither requires nor prohibits the function from producing, in addition, a record of the send observable in the actor's own program — the question is not raised. The formalization can be extended, without modifying any of the three commitments above, to include as output not only the outgoing messages but also a record of the send observable within the actor's own program.

What the canonical formulations establish is the *boundary* between actors — autonomy, isolation, message passing as the exclusive cross-actor mechanism. They do not establish that the boundary be invisible from within. The opacity of the cross-actor send to the sender's program is a separate decision that the canonical sources adopted but did not derive.

4.3 The objection from isolation

The most common objection to making cross-actor sends programmatic is that doing so would violate actor isolation. The objection conflates two distinct properties.

Isolation requires that no actor read or write another's state. Recording a cross-actor send in the sender's journal does not require either. The sender writes to its own journal — its own state. The journal entry describes what the sender did: *"I sent a message of type M to actor B at time T."* The receiver, when it processes the message, writes a corresponding entry to its own journal: *"I received a message of type M from actor A at time T."* Both records are local; neither requires shared state. Isolation is preserved.

What changes is not who can read whose state, but what each actor's program is permitted to say about its own activity. That change is local to the actor's own program; it is invisible across the boundary.

4.4 The objection from orchestration

The second common objection is that recording cross-actor causation as program would introduce orchestration. The objection conflates a record of causation with a director of behavior.

Orchestration is the architectural pattern in which an external coordinator decides what each participating actor does next. The coordinator dispatches commands; the participants execute them. Causation flows from the coordinator outward.

A journal entry that records what an actor did is not a coordinator. It does not direct anything. Each actor still processes its own messages autonomously, decides what to do, and emits its own outgoing messages. The journal preserves what happened; an orchestrator would prescribe what should happen. Recording causation does not introduce a director. It introduces narration, not control.

4.5 Closing

The assumption stated in §3 is therefore not entailed by the actor model. It does not follow from Hewitt's formulation, from Agha's formalization, from the requirement of isolation, or from the absence of orchestration. The opacity of the cross-actor send to the sender's program is contingent in exactly this sense — not a theorem of the model — but it is not arbitrary: it is the shadow of a different objective function (§2.4), well suited to location transparency, decoupling, and fault isolation, and so adopted by the canonical sources and left in place. It is not, in any formal sense, a property the model requires.

The consequence is bounded but decisive: the absence of semantic continuity at actor boundaries is not a necessity of the actor model. It persists because it served other objectives, not because the model demands it — so removing it is a change of objective, available without abandoning a single commitment, rather than the correction of an oversight.

---

5. Consequences

When the assumption stated in §3 is in force, cross-actor causation is held outside any actor's program. The consequences of this displacement are not operational inconveniences; they are structural. Four are examined here. Each is a distinct symptom; each is the same absence — the absence of semantic continuity at actor boundaries — observed from a different angle.

5.1 Auditability requires external reconstruction

The question *"why did this happen?"*, asked of an effect that crossed an actor boundary, cannot be answered by reading any single actor's program. The

sender's journal records what the sender did; the receiver's journal records what the receiver did; neither records the link. To answer the question, an auditor must consult tools that live outside the programs: correlation IDs threaded through messages by the runtime, distributed tracing systems that observe the runtime from outside, log aggregation pipelines that join records across actor boundaries.

These tools work, but their necessity is evidence that the program does not contain what they reconstruct. They do not extend the program; they substitute for it. Auditability therefore depends on infrastructure that compensates for what the program does not record.

5.2 Replay does not reconstruct cross-actor history

Replaying an actor's journal reconstructs that actor's local history: which messages it received, how it responded, what state it produced. Replay of two actors' journals reconstructs two local histories. It does not reconstruct the joint history. The joint history never existed as a program artifact to be replayed.

The reason is direct. The act of one actor sending a message to another is not in either journal as a program statement. The sender's journal contains the operations leading up to the send; the receiver's journal contains the operations following its receipt. The send itself — the event that joins them — has no representation in either record. Replay can therefore reconstruct each actor in isolation, but not the causal chain that connects them.

This consequence matters wherever event sourcing, time-travel debugging, or post-hoc analysis is intended to span more than one actor. The model that promises replayability delivers it within actors and not across them.

5.3 Cross-datacenter replication loses program-level causation

When journals are replicated across data centers — a standard pattern for disaster recovery and geographic distribution — the per-actor histories travel with them. The cross-actor causation does not, because it was never written anywhere that replication can carry. It lived in the dispatcher, the message broker, the runtime — components that are typically per-cluster or per-DC.

Recovery from a DC failure can therefore reconstruct each actor's local state but cannot reconstruct, from the replicated material alone, the causal chain that led to the system's pre-failure state. The chain is reconstructed by replaying the broker's queue, by re-issuing messages that were in flight, or by reconciliation processes that assume the state and reconstruct backward. None of these steps is part of any actor's program; all of them depend on infrastructure-level apparatus that may or may not have been replicated.

5.4 Debugging across actors is log archaeology

A developer asked to explain why an actor produced a particular state, when that state was caused by a chain of events crossing several actors, cannot read a single program to find the answer. The developer assembles the answer from logs, traces, broker dumps, and timestamps — pieces of evidence whose joining is itself an act of reconstruction. The discipline that emerges is log archaeology: the careful inference of a causal narrative from artifacts that are not narrative themselves.

The discipline is real and, in some organizations, mature. Its maturity is proportional to the absence of programmatic causation — the cost is not that the discipline fails but that it must exist at all, reconstructing from infrastructure what no program records. But its existence is the symptom. A program whose causation is recorded as program admits debugging by reading; a program whose causation is dispersed across infrastructure admits debugging only by inference.

5.5 Closing

The four consequences are not separate problems with separate fixes. They are four manifestations of a single structural absence: causation between actors is held outside any actor's program, so any operation that requires reading the cross-actor causal chain — auditing, replaying, replicating, debugging — must reconstruct it from artifacts that lie outside the program.

Symptom	What is lost	What external apparatus reconstructs it
Auditability	the causal chain as part of any program	distributed tracing, correlation IDs, log aggregation
Replay coherence	the joint history reconstructible from per-actor journals	custom event-sourcing layers that span actors
Cross-DC replication	program-level causation when journals cross DCs	broker-level log replication, reconciliation processes
Debugging	the ability to read the cross-actor flow as a single program	log archaeology — manual correlation of timestamps and IDs

The cost of the assumption is not a list of operational difficulties to be addressed by better tools. It is the systematic displacement of causation from program to infrastructure. The tools that arise — tracing, correlation IDs, reconciliation

processes, log archaeology — are not extensions of the program. They are evidence of what the program does not contain.

---

6. Why existing approaches do not dissolve the assumption

The consequences of the assumption named in §3 — auditability through external reconstruction, replay limited to single actors, cross-DC fragility, debugging by log archaeology — are not unrecognized in the actor-systems literature. Each has accumulated a corresponding repertoire of patterns intended to address it. This section examines the principal patterns and shows that they are responses to the consequences, not dissolutions of the assumption that produces them. Each pattern reconstructs cross-actor flow somewhere; none records it as a sentence of any participating actor's program.

This section carries as much of the paper's weight as the instantiation that follows. Once the assumption of §3 is named, sagas, choreography, distributed tracing, and workflow engines stop reading as unrelated tools and resolve into responses to one question — *where does the causal sentence live?* — each answering it the same way: by placing the sentence somewhere outside the participating actors. That so many mature patterns are answers to a single absence is the paper's central claim; §8 is its existence proof.

6.1 Sagas

The saga pattern, in both its orchestrated and choreographed forms, addresses the problem of multi-actor business transactions: an operation that requires several actors to act in sequence, with compensating actions if any step fails.

In the orchestrated form, a saga orchestrator holds a state machine that represents the cross-actor flow. The orchestrator dispatches commands to the participating actors and listens for their responses; if a step fails, the orchestrator emits compensating commands that undo or correct earlier steps. The cross-actor flow is therefore programmatic, but it is the orchestrator's program — not the program of any participating actor. From the perspective of an actor receiving the orchestrator's command, the message is indistinguishable from any other; the actor's program does not know it is in a saga.

In the choreographed form, the orchestrator is dissolved into a protocol. Each participating actor publishes events when it completes a step; the next actor in the flow subscribes to those events and triggers its own step. The cross-actor flow is encoded across the actors as a distributed protocol — but no single actor's program contains the flow. Each actor's program contains its local response to events; the flow itself is the implicit composition of those responses, observable only from outside.

Both forms treat the cross-actor flow as a separate concern from any participating actor's program. The orchestrated form externalizes it to a coordinator;

the choreographed form distributes it across event subscriptions. Neither form makes the cross-actor send a sentence of the sender's program. The assumption stated in §3 is preserved by construction. The saga makes cross-actor flow programmatic only by relocating it outside the actors whose behavior it coordinates. A saga also does more than relocate the record: it implements transactional recovery — compensating actions, retries, step boundaries — a concern that is genuinely hard independently of where the causal record lives. tell does not provide that recovery and is not a substitute for it; what tell addresses is narrower and prior — *where the record of the cross-actor send lives*. The two are different instruments, compared here only on that axis. They compose rather than compete: tell's relocation of the record is the substrate a receiver-side saga can sequence over — each asserted fact a step the saga recognizes, a compensating action itself another asserted fact (`SaleReversed`) rather than a coordinator's command.

6.2 Distributed tracing

Distributed tracing — OpenTelemetry, Zipkin, Jaeger, and similar frameworks — addresses the problem of observability across actor boundaries. A trace context is propagated through messages by the runtime; spans are emitted by participating components; a trace storage backend reconstructs the cross-actor causal chain from the spans, presenting it as a tree or timeline.

The reconstruction is real and useful. It also confirms the structural absence it compensates for. The trace is built from artifacts that exist outside the participating actors' programs: the trace context is metadata threaded through messages by middleware; the spans are emitted by instrumentation that observes the runtime; the trace storage is a separate service that joins the spans. None of these artifacts is part of any actor's program. They are the apparatus that makes the absence navigable.

A trace is also lossy in a way that program is not. A trace records that a span with given attributes occurred; a program records what was said and why. The two are not equivalent. The trace is sufficient for observability; it is not the program of the cross-actor flow. It exists precisely because no such program exists.

6.3 Workflow engines

Workflow engines — Temporal, Cadence, AWS Step Functions, Camunda, Argo Workflows — externalize the cross-actor flow into a dedicated programming environment. The workflow is written as a separate program in the engine's own model; the engine dispatches activities to participating services; the engine's program records the flow as it executes.

Workflow engines are unusual among the patterns considered here because they do produce a programmatic record of the cross-actor flow. The flow is a program — an explicit, replayable, durable program. But the program belongs to the

workflow engine, not to any participating actor. The actors are activities invoked by the workflow; their programs do not contain the workflow.

The asymmetry matters. From the perspective of a participant, an activity invocation is a remote procedure call from an external system; the participant's program records that it received an invocation and produced a result. The relationship between activities — the flow that connects them — lives in the workflow engine, in a separate codebase, in a separate execution model. Reading the participant's program does not reveal the workflow; reading the workflow does not reveal the participant's local program. Two complementary records that do not compose into one. The workflow and the actors form a bipartite description of what, in a semantically continuous system, would be a single program.

6.4 Causal and message logging

A sharper objection comes not from the business-flow patterns above but from distributed-systems fault tolerance, where recording what one process sent another is decades old. Message logging — in its sender-based, receiver-based, and causal variants, surveyed by Elnozahy, Alvisi, Wang, and Johnson (2002) — records the messages a process sends or receives so that a crashed process can be reconstructed by replaying them (Johnson & Zwaenepoel, 1987). Lamport's happened-before relation (Lamport, 1978) and the vector clocks that refine it (Fidge, 1988) record cross-process causal order directly. These mechanisms predate this paper by decades and unambiguously record cross-entity causation. A distributed-systems reader is entitled to ask: how is a *tell* recorded in the sender's journal different from a sender-based message log, which has existed since the 1980s?

The difference is the operational/programmatic distinction of §3, and message logging falls cleanly on the operational side. A message log is maintained by the recovery protocol beneath the application, for the recovery protocol's purposes: it captures message payloads and delivery metadata so the fault-tolerance layer can resend or replay them after a crash, and it is read by that layer, not by any actor's program. The application's program does not contain the act of sending as a sentence; the log is apparatus the runtime keeps *about* the program — the same category as the distributed trace of §6.2, differing in purpose (recovery rather than observability) but not in kind. A vector clock is narrower still: it records the *order* of causation, not its *content* — that B's state causally followed A's, not what A said to B or why.

Tell records the send as a sentence in the actor's program: a dense DSL statement in the journal that *is* the program — the anti-porosity of Paper 1, the externalized parameters of Paper 2 — read as narrative and replayed as program. The contrast is exact. A sender-based message log answers *"what bytes must I resend to recover this process?"*; the journal answers *"what did this actor say, to whom, and why?"* The first is a recovery artifact, payload-oriented

and type-erased; the second is the program. The same holds for event-sourcing causation and correlation identifiers (Young, 2010; Vernon, 2013) and for the process-manager pattern (Hohpe & Woolf, 2003): each threads an identifier or a coordinating state machine through messages so the cross-entity chain can be reassembled downstream — operational reconstruction by the same logic as §6.1–6.3. That cross-entity causation has been captured for thirty years is not in dispute. What no prior mechanism does is record it as a sentence of the sending actor's own program. Message logging is, if anything, the strongest witness for §3's distinction: it is the most mature, most studied apparatus for capturing cross-actor sends, and it lives entirely on the operational side of the line.

6.5 Closing

The patterns examined fall into two groups. Orchestrated sagas, choreographed sagas, distributed tracing, and workflow engines (§6.1–6.3) are responses to the same consequence: cross-actor flow is not in any participant's program, and something must compensate for that absence — a coordinator's state machine, an event-subscription protocol, a trace storage backend, a workflow engine's separate program. Causal and message logging (§6.4) arises for a different purpose — fault-tolerant recovery — but shares the decisive property: the cross-actor send it records lives in a log beneath the program, not as a sentence within it.

Pattern	Where the flow is encoded	Does it preserve the flow as program in the actors?
Saga (orchestrated)	In the orchestrator's state machine	No — externalized to a coordinator
Saga (choreographed)	Across event handlers in each actor	No — only local responses appear
Distributed tracing	In the trace storage, observed by an external system	No — the program is observed, not extended
Workflow engine	In the workflow engine's separate program	No — the workflow is a separate artifact
Causal / message logging	In the recovery layer's message log, beneath the program	No — apparatus for crash replay, not a program statement

The five "No"s share a structure. In every case, cross-actor flow is made programmatic only by placing the program somewhere other than in the actors whose behavior constitutes the flow. The compensation always occurs outside the participants.

These patterns are therefore not solutions to the assumption stated in §3; they are architectural responses that accept the assumption and build systems around

it. Their sophistication, maturity, and widespread adoption are not evidence that the assumption is correct. They are evidence that the absence of semantic continuity is costly enough to justify entire subsystems dedicated to reconstructing what the program does not contain.

---

7. Reformulation

The diagnosis is complete. The assumption stated in §3 — that causation between actors is treated as operational rather than programmatic — is not entailed by the actor model (§4); produces a structural absence of semantic continuity at every actor boundary (§5); and survives in every existing pattern that responds to the resulting consequences (§6). The remainder of the paper takes the diagnosis as established and asks what would have to be the case for the assumption to be dissolved. This section answers that question by deriving the conditions under which semantic continuity is preserved across actors, without violating any of the actor model's structural commitments and without introducing orchestration.

7.1 Three conditions

The conditions are derived directly from §4. The actor model entails autonomy, message-based communication, and isolation (§4.1). The construct introduced in §1 requires that the cross-actor causal chain be recorded as part of some actor's program (semantic continuity). Reconciling the two yields three conditions, each of which addresses one structural commitment of the model and one element of the construct. The conditions are not design choices; they are the consequences of reconciling the model's commitments with the construct.

Condition C1 — Locality of writes. When an actor sends a message to another actor, the sender records the act of sending in its own journal, and only there. The receiver, on processing the message — including an acknowledgment returned for an earlier send — records the receipt in its own journal, and only there. No actor writes to another's state. This condition preserves the model's isolation commitment: each journal is local property; recording cross-actor causation does not require shared state.

Condition C2 — Causation as program statement. The cross-actor send appears as a sentence in the sender's program — a journal entry whose content names the recipient and the message. Reading the sender's journal reveals not only what the actor did locally but also the act of speaking that connected its program to another actor's. The cross-actor edge becomes part of the program's narrative rather than an artifact reconstructed from outside it. The construct introduced in §1 — semantic continuity — is realized at the sender's boundary by this condition; symmetrically, the receiver's program records the receipt as a sentence about who spoke to it.

Condition C3 — No external coordinator. No party outside the participating actors authors what each actor does next — what it does, which message it sends, how it responds. The message-passing layer may carry those messages, with whatever reliability it provides; what C3 forbids is an external *author* of the flow — a coordinator whose state machine chooses each actor's next step — not an external *carrier* of it. Each actor processes its own messages autonomously and decides its own response. The journal records what happened; no orchestrator prescribes what should happen, and no participant outside the actors is required to interpret the record. This condition preserves the absence of orchestration that §4.4 identified as a non-negotiable property of the alternative.

The three conditions are independent. C1 alone preserves isolation but does not establish continuity; C2 alone establishes continuity but, without C1, would risk shared-state writes; C3 alone preserves autonomy but does not address the recording question. Together, the three conditions yield a system in which cross-actor causation is recorded as program in each participant's local journal, dispatched through the existing message-passing layer, and coordinated by no external party.

7.2 *tell* as primitive

A primitive that satisfies the three conditions by construction can be defined.

> Define *tell* as a sentence in an actor's program that, in a single act, (a) records an entry in the sender's own journal naming the recipient and the message, (b) dispatches the message through the existing message-passing layer, and (c) requires no coordination with any party outside the sender and the recipient.

The three components correspond to the three conditions. Component (a) satisfies C1 — the journal entry is written only to the sender's journal — and C2 — the entry is the program statement that names the cross-actor causation. Component (b) inherits the existing message-passing semantics of the actor model; nothing about the dispatch mechanism is new. Component (c) satisfies C3 — the act involves only the sender's local write and the dispatch; no orchestrator participates.

The primitive is conceptual. Its naming follows actor-systems convention: in Akka and elsewhere, *tell* designates a fire-and-forget cross-actor send (§2.3). The collision with Akka's verb is not accidental but argumentative: the verb has been part of actor-systems vocabulary for over a decade alongside the assumption that the verb's effect was extra-programmatic. The primitive defined here keeps the verb and changes what the verb records. Where Akka's *tell* leaves no trace in either actor's program (§2.3), this primitive's effect is to make the trace the program. The difference is not in the dispatch semantics but in what the program is allowed to say about the dispatch.

The canonical effect can be stated in one sentence: *tell* turns the journal into the boundary of causation. The boundary between actors does not disappear; it remains the locus of message passing. What changes is that the boundary becomes inscribed in each participating actor's program.

7.3 What changes, what does not

A reader familiar with the actor model may ask whether the conditions above amount to a different model. They do not. Each of the actor model's three commitments survives unchanged.

Autonomy is preserved. Each actor still processes its own messages serially. *tell* introduces no requirement of synchronization with other actors; the sender does not wait for the recipient. The conditions add a write to the sender's local journal; they do not couple the sender's progression to the recipient's.

Message-based communication is preserved. Actors still communicate by passing messages, exclusively. *tell* uses the same message-passing layer that the actor model already specifies; no shared memory, no remote procedure calls, no synchronous read of another actor's state. The dispatch mechanism is unchanged; only the recording at each end is added.

Isolation is preserved. No actor reads or writes another's state. The sender's journal entry is written by the sender. The receiver's journal entry, if any, is written by the receiver. The two records are local; they reference each other by content, not by shared address.

What changes is what each actor's program is permitted to say about its own activity (echoing §4.3). What does not change is what each actor can do, observe, or share. The opacity of the cross-actor send to the sender's program — which §3 named as the assumption and §4 demonstrated to be contingent — is the only thing that the conditions remove. The actor model remains intact; only the historical interpretation of what must remain invisible to the program is removed.

7.4 The shape of the act

The structural shape of *tell* is rendered below. The diagram is included not to introduce new content but to make the relations among the components of §7.2 visible at a glance. The diagram is deliberately mundane: nothing new is introduced except the presence of the journal entries.

```
sequenceDiagram
    participant ProgA as Actor A's program
    participant JA as A's journal
    participant MPL as Message-passing layer
    participant ProgB as Actor B's program
    participant JB as B's journal
```

```
ProgA->>JA: tell msg to B — record the assertion
ProgA->>MPL: dispatch msg
MPL->>ProgB: deliver msg
ProgB->>JB: process and record receipt entry
```

Two journal writes (in JA and JB), one dispatch through the existing message-passing layer, no participant outside the sender and the recipient. The diagram reproduces the three conditions visually: writes are local (C1), the cross-actor flow is recorded as program at both ends (C2), and no coordinator is present (C3).

7.5 Closing

The conditions are not extensions of the actor model; they are the conditions under which the assumption named in §3 can be removed without altering the model's structural commitments. *tell* is one realization of the conditions; any realization that satisfies C1, C2, and C3 would dissolve the assumption identified in §3. But the conditions presuppose a substrate: C2 requires the cross-actor send to be a *statement of the program* — a dense operation, not a serialized event (§3) — which is the anti-porosity of Paper 1 and the externalized parameters of Paper 2. The generality is therefore over primitives *on a journal-as-program substrate*, not over actor frameworks in general; a framework that records sends as opaque payloads would satisfy C2 only by first adopting that substrate, which is the subject of Papers 1–2 and beyond this paper's scope.

The reformulation is conceptual. The remainder of the paper presents the empirical question: can the conditions be realized in a running system? §8 answers by exhibiting an instantiation and demonstrating it through an illustrative case study.

---

8. Instantiation

8.0 Origins of the instantiation

The instantiation discussed below is provided by Puppeteer, an actor-based framework whose journal-as-program substrate predates the analysis presented in this paper. The framework's lineage runs from a 2005 autopersistence prototype — domain classes that persisted themselves through reflection, with no schema decisions in the domain code — through a DSL that emerged as the persistence substrate, to event sourcing as the runtime's primary discipline. By the time the conditions of §7 were articulated, the framework already satisfied them; the present section reads as observation of that alignment, not as derivation of the framework from the conditions. This invites a fair question of circularity — were C1–C3 read off Puppeteer and then presented as model-derived? They are not: §7.1 derives them from the actor model's three commitments and the construct alone, citing no framework feature. That a framework built for other

reasons, years earlier, already satisfies them is therefore independent evidence that the model-derived conditions are satisfiable — not a sign they were retrofitted.

8.1 The case study domain

The case study is a simplified loyalty domain modelled on a production scenario from a deployment of the framework. A Seller actor confirms a purchase order via a domain command. A RewardEngine actor holds a registry of campaigns; for each purchase event, the RewardEngine evaluates which campaigns qualify (by date and amount thresholds) and applies the corresponding rewards to the customer.

The flow is intentionally minimal: one actor produces a domain event, another reacts to it, and the relationship between the two needs to cross an actor boundary. The simplicity is deliberate — the case study illustrates a structural property of the cross-actor mechanism, not the complexity of any particular business workflow.

8.2 The instantiation: *tell* in Puppeteer

Puppeteer's Reaction surface exposes three planes — *Program* for in-actor read-only effects, *Metadata* for journal-metadata changes, and *Causation* for cross-actor causation. The three planes name what the verb touches; *tell* lives on the third.

The Seller's Reaction is defined as follows:

```
seller.Reactions.DefineReaction("PurchaseFunnelToRewards")
    .Job().Company()
    .WithSharedHydration()
    .Seek("Purchase")
      .OnMatch("[s:Seller].purchase($order, $date, $amount, $customer)")
    .Causation.Continue(@"
        tell PurchaseConfirmed
            with @order, @date, @amount, @customer
            to RewardEngine('rewards-1')
            once @order;
    ");
```

The `tell` statement is a sentence in the actor's DSL, and the kind of sentence matters: the Seller *asserts a fact it has lived* — `PurchaseConfirmed`, named in the Seller's own vocabulary — addressed to the RewardEngine (`to RewardEngine('rewards-1')`), carrying the values that fact involved (`with @order, @date, @amount, @customer`) under a per-utterance identity taken from the order it carries (`once @order`), so one compiled action issues a distinctly-identified assertion for each purchase. It does not invoke a method on the RewardEngine, and it does not name how the message travels. This follows the single discipline

the journal obeys throughout this series: an actor's program may record only what that actor could itself have said. The Seller can say *that a purchase was confirmed*; it cannot say *how the RewardEngine applies rewards* — that is the RewardEngine's verb, recorded in the RewardEngine's own journal — nor *which broker carries the message*, which is deployment, not program. The Reaction is established once and thereafter watches the Seller's journal; when the domain command `s.purchase(...)` lands as an entry, the standing Reaction matches it and its `.Causation.Continue(...)` body fires, journaling the assertion on the Seller's side.

The assertion is journaled as a typed *message-action* — defined once (its signature deduced from the values it carries) and then invoked. This is the same define-then-invoke shape *every* parameterized operation takes on the dense journal-as-program substrate this series builds on (Papers 1–2): the domain purchase, issued with typed parameters (`date`, `amount`), is journaled the same way, so the shape below appears twice — once for the purchase, once for the assertion. After the bridge delivers the assertion to the RewardEngine and the receiver acknowledges, the Seller's journal contains six entries:

```
[0]  s = Seller();
[1]  (define of the message-action s.purchase — its typed signature)
[2]  s.purchase('ord-100', 5/9/2026, 250, 'cust-42');
[3]       (define   of   the   message-action   PurchaseConfirmed   —
its typed signature)
[4]  tell PurchaseConfirmed
         with 'ord-100', 5/9/2026, 250, 'cust-42'
         to RewardEngine('rewards-1')
         once 'ord-100';
[5]  tell ack 'ord-100' from RewardEngine('rewards-1');
```

The values are passed as typed parameters, not interpolated into the script text — the definition records the signature `s.purchase(order, date, amount, customer)` and the invocation records the bound arguments, so the journal stores the operation and its values separately, exactly as the tell does. The tell's identity is the order it carries (`once @order`), which the invocation binds to `'ord-100'` — the ack in entry [5] returns under that same key. The tell's signature is inferred once — at first use, on the live path — and recorded in entry [3]; replay replays that recorded definition rather than re-inferring it, so the message-action is identical and order-independent across replays. This is the define-and-invocation discipline of Paper 2, under which a name is defined once and thereafter invoked; a reused name resolves as any operation does there. The inference is a write-time convenience, never a replay-time computation.

The three conditions of §7 are realized by these entries.

- **C1 (Locality of writes).** The six entries are in the Seller's journal only. The RewardEngine's journal records its receiving operation independently — the two journals never share storage.

- **C2 (Causation as program statement).** Entries [3]–[4] are the cross-actor assertion rendered as a DSL sentence — defined once as a typed message-action, then invoked. Reading the Seller's journal alone reveals that it asserted `PurchaseConfirmed` to RewardEngine, with what values, at what time. Entry [5] closes the round-trip with the receiver's acknowledgment.
- **C3 (No external coordinator).** C3 forbids an external party that *authors* the flow — one that decides, from its own state, what each actor does next. It does not forbid a *carrier*. The Seller's own program makes the assertion — it records, as a sentence of its program, that it told the RewardEngine and what — and the bridge only carries that assertion onward. The journal names no transport at all: which broker or protocol carries the message is a deployment binding resolved outside the program, not a sentence in it (see below). A saga coordinator is the opposite: its state machine authors the flow, which then lives as a quasi-domain artifact outside every participant's program, and folding it into a participant would dirty that domain with coordination it should not carry. Giving delivery a clean home in the carrier is what lets each actor's journal hold only the causal record — the fact that it spoke. (The reproducibility lab exhibits this edge two ways: a single-process didactic run in which the bridge also stands in for the receiver's own processing, and a separated-receiver run in which a pure in-process broker carries the envelope while the RewardEngine runs its own consumer — mapping the asserted message to a command it owns, journaling that command in its own journal, and acknowledging autonomously. The second instantiates the carrier/receiver split this clause turns on, with no party standing in for the receiver; in a deployment the delivered message is processed by the recipient's own program in the same way.)

A note on entry [5]. The acknowledgment is not the RewardEngine writing into the Seller's journal. The RewardEngine emits an ordinary result in its own journal; the transport routes an acknowledgment envelope back to the Seller; the Seller's own handler processes that inbound envelope and records `tell ack ...` in the Seller's journal. For the ack message the Seller is the *receiver*, so C1's receiver clause applies unchanged — an actor records its own receipt in its own journal. The cross-actor edge is two messages, the assertion outbound and the ack returning, each narrated by the actor that processed it; neither actor writes to the other's journal.

Delivery and correlation are separated by design. The journal is the durable record of what was asserted and what was acknowledged — correlation; redelivery, timeout, dead-lettering, and backoff belong to the transport. Routing, too, lives outside the program: the sentence names the addressee by its logical role (`to RewardEngine('rewards-1')`), and a deployment-level binding — not a clause in the journal — resolves that role to a physical route (which transport, which topic). The journal therefore reads identically no matter which transport that binding selects; the wire never enters the actor's voice. The delivery guar-

antee is whatever the chosen transport provides; the journal's correlation role is invariant across all of them. If an acknowledgment is lost, the transport's retry redelivers and the assertion's identity (`once @order` — an author-chosen key, here the order the assertion carries — or a content hash when `once` is omitted) is the receiver-side deduplication key, so a redelivered tell does not duplicate the effect; a replayed journal re-executes the assertion — rebuilding the in-flight record — but its dispatch to the transport runs only on live execution, so replay does not re-send (the transport never sees a message from rehydration). A crash in the narrow window between journaling the assertion and dispatching it strands the envelope — journaled as issued, yet never handed to the transport; on recovery the rehydrated sender reconstructs that pending tell from the journal alone and the transport, the sole authority on delivery, testifies its fate, which the sender records as a verdict in its own voice (§8.5 G4) — so a crash no longer leaves the journal asserting a send that never landed: each issued tell is resolved (acknowledged, or marked unacknowledged by its addressee) or honestly left pending for the transport to settle, bounded by what that transport can still testify. The delivery model is thus at-least-once with identity-based deduplication; the consistency machinery it rests on — the now/deferred partition and its deduplication discipline — is developed in Paper 3, on which this construction builds.

The framework rejects `tell` outside `.Causation.Continue(...)`. A direct `tell` from a top-level command throws a runtime exception, with the error message pointing the developer at the correct surface. The cross-actor send is always a sentence in some Reaction's Causation body, never a free-floating call.

One feature of the sentence repays attention: the message is named in the past — `PurchaseConfirmed`, a fact the Seller has already lived — never in the imperative. This is not a style convention but a consequence of the criterion: an actor can assert only what has already happened in its own history, and by the time the assertion is journaled the purchase is a settled entry. A *command* — the directive at a request edge, which may still be refused — is named in the present imperative (`ApplyReward`, `Confirm`); to name a command in the past would be a category error, an order to do what is already done. The tense is the surface mark of which kind of act the sentence is.[1] This is where the construct parts from the canonical actor frameworks: Akka exposes a single send verb (`!`, *tell*) for every message and accordingly advises naming them all in the past tense — folding the report of a fact and the issuing of a command into one form, which is right for the events and wrong for the commands, because one verb cannot tell them apart. Here the two are kept distinct and the tense marks the difference — the assertion the sender makes about its own past, and the command the receiver runs in its own present, landing in two journals, in two

[1]The surface mark is English verb tense because the DSL is written in English; the distinction it carries — asserting a settled fact versus issuing a directive — is not itself tied to tense, and a language that grammaticalizes aspect rather than tense would mark the same distinction by other means. The claim is about the two kinds of act, not about tense as a linguistic universal.

voices.

8.3 Three implementations side by side

The same domain — the Seller confirms a purchase, the RewardEngine applies qualifying campaigns — is exercised in three styles. The code that distinguishes each style and the journals each produces are reproduced below without commentary. The interpretation follows in §8.4.

The saga and choreography implementations are the author's own, written for this illustration and deliberately kept minimal (Appendix A); they are not independent or performance-tuned baselines. The illustration is accordingly structural, not competitive: it shows *where* each style records the joint cross-actor history, not that tell is faster, more resilient, or operationally simpler. Those are measurable dimensions this paper does not test (§1).

Its weight is definitional, not empirical. The alternatives are written in their canonical shape — an orchestrated saga whose participants are commanded by a coordinator that owns the flow, and a choreography whose actors publish to a shared bus — so the reader can see in journals what each pattern's defining feature entails for where the joint history lands. The obvious objection — that a saga could have each participant record its own role — names exactly the tell move: a participant that records its cross-actor participation as a statement of its own program has, by that act, satisfied C2. The contrast is therefore definitional: the alternatives place the joint history outside any participant's program because their defining feature *is* externalized coordination. The journals below illustrate that truth; they are not measured evidence that one pattern outperforms another, and a differently-built saga would not change where externalized coordination, by definition, leaves the record.

Style 1 — Saga (orchestrated) A SagaCoordinator actor drives the workflow via direct commands to participants:

```
saga.Using("step = 'PurchaseRequested';").PerformCommand();

seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
      .PerformCommand();

saga.Using("step = 'PurchaseConfirmed';").PerformCommand();
rewards.Using(@"
```

```
    foreach (c in loyalty.Campaigns()) {
     if (c.Applies(@date, @amount) == true) { c.Reward(@order, @customer); };
    };")
       .WithParameters(p => {
           p["order",    typeof(string)]   = "ord-100";
           p["customer", typeof(string)]   = "cust-42";
           p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
           p["amount",   typeof(decimal)]  = 250m;
       })
       .PerformCommand();

saga.Using("step = 'RewardsApplied';").PerformCommand();
```

Every value crosses the DSL boundary as a typed parameter — `p["date", typeof(DateTime)]`, `p["amount", typeof(decimal)]`, the string ids too — bound through `.WithParameters(...)`, never string-interpolated into the script text. Each command is issued separately, so each lands as its own journal entry; a parameterized command is journaled as a message-action (define-then-invoke), and two campaigns added by the *same* parameterized command share one definition. The RewardEngine holds two campaigns — `C-newcomer` (min 10, qualifies for the 250 purchase) and `C-highroller` (min 1000, does not) — so the receiver's loop applies one of the two. After the run, the three journals contain:

```
SagaCoordinator's journal (3 entries):
  [0] step = 'PurchaseRequested';
  [1] step = 'PurchaseConfirmed';
  [2] step = 'RewardsApplied';

Seller's journal (3 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');

RewardEngine's journal (6 entries):
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
 [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };
```

Style 2 — Choreography (event-driven, no coordinator) An event bus mediates the cross-actor handoff:

```
bus.Subscribe(ev => {
    if (ev.StartsWith("PurchaseConfirmed:")) {
```

```
        rewards.Using(@"
            foreach (c in loyalty.Campaigns()) {
           if (c.Applies(@date, @amount) == true) { c.Reward(@order, @customer); };
            };")
          .WithParameters(p => {
              p["order",    typeof(string)]   = "ord-100";
              p["customer", typeof(string)]   = "cust-42";
             p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
              p["amount",   typeof(decimal)]  = 250m;
          })
          .PerformCommand();
    }
});

seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
      .PerformCommand();
bus.Publish("PurchaseConfirmed:ord-100");
```

After the run:

```
Seller's journal (3 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');

RewardEngine's journal (6 entries):
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
 [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };

Bus log (1 entry):
  published: PurchaseConfirmed:ord-100
```

Style 3 — Tell (Puppeteer) A Reaction on the Seller observes its own purchase and issues a `tell` from its `.Causation.Continue(...)` body:

```
seller.Reactions.DefineReaction("PurchaseFunnelToRewards")
    .Job().Company()
    .WithSharedHydration()
    .Seek("Purchase")
      .OnMatch("[s:Seller].purchase($order, $date, $amount, $customer)")
    .Causation.Continue(@"
        tell PurchaseConfirmed
            with @order, @date, @amount, @customer
            to RewardEngine('rewards-1')
            once @order;
    ");

seller.Reactions.Execute();  // once defined, the Reaction stands over the journal and applies
                                          // to every matching entry —
it is not invoked per command. A Job is
                    // swept on demand (driven here for a deterministic test); a Cue
                         // Reaction runs continuously on its own thread.
seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
      .PerformCommand();
// the standing Reaction observes the purchase and asserts PurchaseConfirmed to RewardEngine;
// the bridge delivers and acks back
```

The tell's identity is `once @order` — the per-utterance key *is* the order id, so one compiled action issues a distinctly-identified assertion per purchase, and the ack returns under that same id.

After the run:

```
Seller's journal (6 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');
  [3] (define of the message-action PurchaseConfirmed)
  [4] tell PurchaseConfirmed
        with 'ord-100', 5/9/2026, 250, 'cust-42'
        to RewardEngine('rewards-1')
        once 'ord-100';
  [5] tell ack 'ord-100' from RewardEngine('rewards-1');

RewardEngine's journal (6 entries):
```

```
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
  [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };
```

8.4 Structural reading

The three implementations in §8.3 exercise the same logical flow but record it differently. Three pairs of journals were exhibited.

Saga: the SagaCoordinator's journal contains the workflow narrative — `PurchaseRequested → PurchaseConfirmed → RewardsApplied`. The Seller's journal records its local purchase only; it does not know it is part of a saga. The RewardEngine's journal records its local reward only; equally unaware. Three journals, three local stories. Only the coordinator's journal contains the joint history.

Choreography: no coordinator exists. The Seller's journal records the local purchase; the publish to the bus is invisible to the actor. The RewardEngine's journal records the local reward; the receipt from the bus is invisible to the actor. The bus's own log records the publish. No actor's journal contains the joint history; the bus log, an external infrastructure artifact, is the only place the cross-actor handoff is recorded.

Tell: the Seller's journal contains the purchase, the assertion (journaled as a typed message-action — defined once, then invoked), and the ack — six entries in all (each parameterized operation is a define plus an invocation) that constitute the joint history as a sequence of DSL sentences. The RewardEngine's journal records its local reward, as in the other styles. Under tell, the sender's journal alone reconstructs this cross-actor edge — the single hop from Seller to RewardEngine; §8.5 examines what happens when the chain is longer.

The three styles produce equivalent business outcomes. They differ structurally in where the cross-actor causal chain is recorded. The difference is not cosmetic. The saga coordinator, the event bus log, the distributed trace, and the workflow engine all exist to compensate for the absence identified in §3. Under tell, that program-level absence is gone: the sender's journal already contains the cross-actor narrative those patterns reconstruct elsewhere, so the apparatus built to recover it has nothing to recover.

The claim is about the *record*, not the wire. Tell does not eliminate the message-passing infrastructure: delivery remains operational, carried by whatever transport a deployment-level binding resolves the addressee to (§8.2), and the bridge that routes envelopes is part of it. What moves into the program is the causal record — the assertion the sender makes, and the acknowledgment of receipt — not the act of delivery. The contribution is the relocation of the narrative into

the program, not the removal of transport.

Style	Joint history location	Audit path
Saga (orchestrated)	In the saga coordinator's journal exclusively	Read the coordinator's journal; participants' journals are half-stories
Choreography (event-driven)	In no actor's journal; the bus log is the only joint artifact	Merge participants' journals via correlation id, plus the bus's log
Tell (program-level cross-actor primitive)	In the sender's journal as a sequence of DSL sentences	Read the sender's journal entries [tell] and [ack] verbatim

In the first two styles, an additional architectural element is required to make the cross-actor flow observable as a narrative: a coordinator, a bus log, a trace backend, or a workflow program. In the tell style, no additional element is introduced. The narrative exists where the actor model already records program: the journal.

8.5 Property validation

Four property tests probe the claims of §5 and §6, all reproduced by the harness in `labs/lab04-tell` run against the public commit recorded under Code provenance. The first three (G1–G3) demonstrate that consequences claimed in §5 — auditability through external reconstruction, replay limited to single actors, cross-DC fragility — are reversed under tell, exhibiting properties that would be inaccessible if the assumption named in §3 were in force; they are intentionally chosen to mirror the compensating patterns of §6, each exhibiting a property that, under the assumption of §3, requires an external architectural pattern to achieve. A fourth (G4) turns to a sharper, adversarial question: whether the record stays honest when a tell never crosses.

G1 — Replay coherence (closes §5.2). The first test stages an in-flight tell: the envelope leaves the Seller but the bridge does not deliver it before the test asserts. A fresh actor instance with the same name, no shared transport, no live receiver, and no in-memory state replays the journal alone. Replaying the journal re-executes the tell statement; under replay it rebuilds the in-flight record rather than re-dispatching, so the replayed actor reconstructs the cross-actor state from the journal alone — the program re-ran, with no duplicate send. The joint history exists as a program artifact and replay reaches it.

G2 — Cross-DC replication (closes §5.3). The second test replicates the Seller's journal entry-by-entry to a fresh actor in an independent storage tier — the analogue of moving across data centers. The replicated actor, with no transport connectivity to the original receiver, reconstructs the dedup state

from the replicated bytes alone. The cross-actor causal chain travels with the replication because it was always recorded in a place replication can carry.

G3 — Audit query (closes §5.1). The third test asks the audit question — *why did this happen?* — by reading the Seller's journal directly. The cross-actor assertion is entry [4] (the message-action invocation); the acknowledgment is entry [5]. The cause-effect chain is reconstructed without distributed tracing, correlation IDs, or log aggregation.

Each of these properties is a consequence of cross-actor causation being recorded as program. Under saga, choreography, tracing, or workflow approaches — where it is not — the same properties are reachable only by consulting artifacts outside the participating actors.

G4 — Tell-fate recovery, the honest record under crash. G1–G3 stage an in-flight tell and show the *record* survives a crash, a move across data centers, an audit. A sharper question is what the record *says* when the tell never crosses. The send is journaled before the post-commit dispatch hands its envelope to the transport; a crash in that window strands the envelope — journaled as issued, never delivered, never acknowledged. Left unaddressed this is the one place the "sender's journal alone" property could lie: the journal would assert a send that did not happen. The fourth test stages exactly this crash and rehydrates the sender. Replay reconstructs the set of *pending* tells — issued, neither acknowledged nor settled — from the journal alone; for each, the transport, the sole authority on delivery, testifies its fate and the sender records the verdict *in its own voice* — `tell 'ord-100' unacknowledged by RewardEngine` when the transport reports the envelope failed, the ordinary `tell ack ...` when it reports the envelope was delivered and only the acknowledgment was lost, and nothing while the fate is still in flight (the transport keeps ownership). The verdict names the addressee the Seller did not hear back from, not the broker that carried the message: A may say *"RewardEngine never acknowledged"* — a fact within its own universe — but not *"per the broker that carried it,"* which is infrastructure it has no standing to assert. This is the dual of §6.4's causal and message logging: there, the record of what crossed lives in a recovery layer beneath the program, consulted only by crash-replay machinery; under tell it is a sentence in the sender's program, so recovering it is reading the journal, not excavating infrastructure. After recovery the sender's journal no longer asserts a send that never landed: each issued tell is either resolved — acknowledged, or marked unacknowledged by its addressee — or honestly carried as still pending, for the transport to settle when it can. The guarantee is honesty about the outcome, not omniscience: a transport that cannot testify — one whose own record of a tell's fate did not survive the failure — answers `InFlight`, and the tell stays pending rather than acquiring a fabricated verdict. Delivery stays the transport's; the verdict is the journal's.

The multi-hop limit — an adversarial case. G1–G3 are duals of the consequences named in §5, and so are chosen to show what tell does well. The honest counter-case is a longer chain. The case study is a single hop — the Seller

tells the RewardEngine. Suppose instead a chain assembled the way every hop is: a Reaction on A tells B; B carries its own Reaction that observes the entry its receipt produces and, from that Reaction's `.Causation.Continue` body, tells C. Nothing propagates the chain automatically — each actor opts in with its own Reaction (the envelope's causal identifier is per-hop-local, not a threaded chain id), so the hops remain autonomous, as C3 requires. Each hop is recorded as program in the journal of the actor that originated it — A's journal holds the A→B tell and its ack, B's journal holds both its receipt of A and the B→C tell it issued, C's journal holds its receipt of B. No single journal holds the whole A→B→C chain; reconstructing it end to end means composing A's and B's journals (linkable by envelope identifier across them). In this, tell inherits a distributed-history property of the kind §6.1 identified in choreography — but the difference is in kind, not in absence. Under choreography no participant's journal records the cross-actor edge as program at all; the chain lives only in the bus log and is reconstructed from non-programmatic artifacts. Under tell every edge is a programmatic record in its sender's journal, so the multi-hop chain is a composition of programs — each hop locally complete, auditable, and replayable on its own. Tell makes each edge programmatic and local; it does not centralize a multi-hop chain into one journal. The "sender's journal alone" property (§8.4) is therefore a per-edge guarantee: it holds for the hops an actor originates, which is the whole chain only when the chain is a single hop.

8.6 Closing

The case study and the defensive tests together constitute the existence proof. The conditions of §7 are realizable: a system in which the cross-actor send is recorded as a sentence in the sender's program, dispatched through the existing message-passing layer, and coordinated by no external party can be built and exercised. The instantiation in Puppeteer is one such system. The framework's own production deployment, which the case-study domain is modelled on (§8.1), is proprietary and is not exhibited here; the existence proof this paper offers is the runnable instantiation above. Other realizations of the conditions are possible (§7.5); the present section establishes only that at least one is.

---

9. Relation to previous work in this paper series

The journal exhibited in §8.3 — a sequence of DSL sentences in the sender's program, recording the cross-actor send and its acknowledgment — required several structural preconditions to be a viable substrate. The journal had to be dense rather than porous: filled with operations, not with type-erased payloads. It had to record the operations with their parameter references intact, not with values inlined as literals. It had to maintain a discipline that separates immediate from deferred work, with a guardian for the boundary between them. Each precondition has been the subject of prior structural analysis in the present series.

Paper 1 introduces *porosity* — the representational sparsity that arises when domain state is recorded as serialized data structures rather than as programmatic operations. Anti-porosity is the design principle that the journal records what was said, not what was stored. Without that property, the entries the reader saw in §8.3 — `tell PurchaseConfirmed with ... to RewardEngine('rewards-1')` — could not be programmatic at all; they would be opaque payloads.

Paper 2 introduces *externalized parameters* as the structural precondition under which compilation, caching, and dense journaling become possible at all. Without externalized parameters, the journal could not record what was said with parameter references intact — values would be inlined as literals, or the script would lose its connection to the actor's symbol table. The tell sentence in §8.3 carries `@order`, `@date`, `@amount`, `@customer` as references rather than literals; this paper is what makes that representation possible.

Paper 3 introduces the *partition* between immediate and deferred work, with Reactions as the guardian of the boundary. Paper 3 names the *Causation* plane — `.Causation.Continue(...)` — as the third surface a Reaction may touch (Paper 3 §6.5), and records (Paper 3 §6.8) that the cross-actor case an earlier draft had listed as a limitation is resolved by that plane, while leaving the primitive's full treatment out of scope. The Reaction surface that Paper 3 establishes is the surface on which `tell` lives: the `.Causation.Continue(...)` body the reader saw in §8.3 is where the cross-actor send is permitted to appear. The present paper is the treatment Paper 3 deferred.

The series, taken together, defends a single architectural property under different framings:

> *Puppeteer preserves semantic continuity inside an actor. Tell preserves semantic continuity across actors.*

The first sentence is the joint contribution of Papers 1 through 3: the structural conditions under which the journal can serve as a program-level substrate for what an actor does. The second sentence is the contribution of the present paper: the cross-actor extension of that substrate that the reader saw exhibited in §8.3.

Read as a sequence, the series traces a single thread — the operation. Paper 1 establishes that operations precede state; Paper 2, that they carry their own parameters and can author themselves; Paper 3, that they can generate further operations as reactions; and the present paper, that an operation can be addressed, as an assertion, to another actor. *Tell* is the cross-actor extension of the journal-as-program substrate the earlier papers build, not messaging added to it from outside.

Papers 1–3 are self-deposited preprints on Zenodo (Rivera, 2026a, 2026b, 2026c) and have not undergone peer review, as is the present paper. This paper rests structurally on them — C2 presupposes the substrate Papers 1 and 2 establish (§3, §7), and the Reaction surface on which `tell` lives is Paper 3's (§8.2) — so it

should be read as the latest in a preprint chain, not as resting on peer-reviewed foundations.

---

10. Conclusion

The actor model has, for fifty years, treated cross-actor causation as an operational concern of the runtime rather than as a construct of the program. The treatment was productive: actor systems became fault-tolerant, scalable, and reliable precisely because the separation between an actor's program and the message-passing layer was operationally effective. Out of that productivity grew the ecosystem of patterns analyzed in §6 and exhibited in §8.3 — saga orchestrators, choreography buses, distributed tracing, workflow engines. Each compensates, in its own way, for the program-level absence the reader saw in the journals.

The present paper observes that the absence is not entailed by the actor model. It is a contingent design decision adopted by the canonical sources and propagated through the lineage as its default frame — productive enough that the question of whether the send could be a statement of the program was seldom raised. The conditions under which the absence can be removed — locality of writes, causation as program statement, no external coordinator — preserve every structural commitment of the actor model. A primitive that satisfies the three conditions, whether named *tell* or otherwise, makes the cross-actor send a sentence in the sender's program; the apparatus that exists to reconstruct that narrative after the fact has nothing left to reconstruct. The message-passing layer itself remains — delivery stays operational — but the causal record no longer lives outside every actor's program.

The contribution of this paper is conceptual. The instantiation is the existence proof; the existence proof is the journal the reader saw in §8.3.

The question that opened the paper has, under tell, a different answer than the one the field gave by default: *where does the causal sentence live?* — in the sender's own program, as a sentence the journal records and replay re-runs.

Fifty years of convention are not fifty years of necessity. The actor model does not need to be replaced. Its commitments — autonomy, message-based communication, isolation — survive the reformulation intact. What changes is the historical interpretation of what must remain invisible to the program. The interpretation, named explicitly in §3, examined for contingency in §4, traced through its consequences in §5, shown to require an entire ecosystem of compensating patterns in §6, and contrasted with the alternative in §8.3, is the only thing the present paper asks the field to revise.

Appendix A. Code references

The labs cited in §8 are publicly available in the Puppeteer codebase. The references below are to the test suite of the framework's repository.

Cross-actor primitive surface — `Puppeteer/EventSourcing/Follower/`

File	What it shows	Cited in
`Reaction.cs`	Reaction class, Action terminator dispatch, replay-safe action execution	§7.2, §8.2
`Planes.cs`	The three plane types (`ProgramPlane`, `CausationPlane`, `MetadataPlane`) and their property accessors	§8.2
`ReactionEngine.cs`	Pattern matching surface, `.OnMatch(...)`, plane passthroughs	§8.2

Reproducibility lab — `labs/lab04-tell/`

The runnable harness that reproduces every journal exhibited in §8 ships with this paper (and in `paper04-data.zip`), built against the public runtime commit recorded under Code provenance. The framework's own end-to-end tests for the same scenarios live in the private fork and are not part of the public clone; this lab is the public, self-contained equivalent.

File	What it shows	Cited in
`Program.cs`	The harness, one method per scenario: the three-style side-by-side case study (orchestrated saga, event-driven choreography, tell — same domain, three journal locations of the joint history); G1 replay coherence; G2 cross-DC replication; G3 audit query; G4 tell-fate recovery across the crash window; and the negative gate for `tell` outside `.Causation.Continue(...)`. Prints each actor's journal and a PASS/FAIL line per assertion.	§8.2, §8.3, §8.4, §8.5
`LoyaltyDomainStubs.cs`	Domain-side stubs for `Seller`, `RewardEngine`, `Campaign` — kept minimal so the focus remains on the cross-actor mechanism	§8.1

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `37ad9cf` (2026-07-05). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:2b3ef9f68cf6e3a7a8e32ec73121122cc2ac74c5;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:37ad9cf44e7749f8fa0d2da8cc0c1093ad5a7e83
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future

commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

Appendix B. Bibliography

Agha, G. (1986). *Actors: A model of concurrent computation in distributed systems*. MIT Press.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors* [Doctoral dissertation, Royal Institute of Technology (KTH)]. https://erlang.org/download/armstrong_thesis_2003.pdf

Bernstein, P. A., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Technical Report MSR-TR-2014-41). Microsoft Research.

Elnozahy, E. N., Alvisi, L., Wang, Y.-M., & Johnson, D. B. (2002). A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, *34*(3), 375–408.

Fidge, C. J. (1988). Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference* (pp. 56–66).

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, *30*(3), 611–642.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI-73)* (pp. 235–245).

Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, *21*(8), 666–677.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Johnson, D. B., & Zwaenepoel, W. (1987). Sender-based message logging. In *Proceedings of the 17th International Symposium on Fault-Tolerant Computing*

(FTCS-17) (pp. 14–19).

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, *21*(7), 558–565.

Lightbend. (n.d.). *Akka core: Interaction patterns* [Akka documentation]. https://doc.akka.io/libraries/akka-core/current/typed/interaction-patterns.html (Internet Archive snapshot, 2026-02-16: https://web.archive.org/web/20260216052600/https://doc.akka.io/libraries/akka-core/current/typed/interaction-patterns.html)

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20740697

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20792156

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf